Loops

```
used for repetitive tasks
```

While Loop

execute a block of statements until condition is satisfied
while condition1:
code
else:
code

Break and Continue

```
break -> to get out/stop the loop
continue -> skip current iteration, skip to next
```

For Loop

```
to iterate over a sequence(list, tuple, dict, set, string)

- for iterator in string:
    # apply to other sequences
- for key, value in dict:
- for iterator in dict:
    # gives you keys of the dict
```

Range Function

```
range(start, end, step)
    or
range(end)
    or
reversed(range())
    #to reverse the range
```

Nested For Loop

```
for iterator in sequence:
    for i in iterator:
else:
```

Pass

```
use it to replace empty statement to avoid errors
for iterator in sequence:
    pass
```